Session 5: Thresholding, Morphology, Edges, Watershed

Notebook: `Session5.ipynb`. This is the **most important session for the UGV project**, because the flag detector is essentially *threshold → clean up with morphology → find the biggest blob*.

Order used here: this follows the notebook's order, which is already logical.

1. Thresholding: color, histogram, binary, Otsu
2. Morphology: erode, dilate, open, close, top-hat
3. Edge detection recap (Sobel, Canny)
4. Region segmentation: watershed

> ⚠️ The folder is `Images/` with a capital I. On Windows that doesn't matter, but on Linux or Mac it does.

---

1. Thresholding

💡 **Thresholding** turns an image into a **binary mask**: 255 (white) where a pixel meets a condition and 0 (black) where it doesn't. Almost every detection pipeline starts like this.

Cell 3: imports and `show()`

This is the same `show()` helper as in Session 2. It converts BGR → RGB for color images and uses a gray colormap for 2-D ones.

Cell 4: loading

```
image = cv2.imread('Images/disks.JPG')
rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

- `rgb` is used for plotting **and** for thresholding in the next cell.

Cells 5–6: color thresholding with `inRange` 🎯

```
##      [R,  G,  B]
lower = [100, 15, 17]
upper = [200, 56, 50]
lower = np.array(lower, dtype="uint8")
upper = np.array(upper, dtype="uint8")
mask = cv2.inRange(rgb, lower, upper)
segmented = cv2.bitwise_and(rgb, rgb, mask=mask)
```

- `cv2.inRange(src, lower, upper)` checks **every channel** of every pixel:

    - if `lower[c] ≤ pixel[c] ≤ upper[c]` for **all** channels c (the bounds are inclusive), the mask pixel is 255;
    - otherwise it's 0.
    - The output is a **single-channel** `uint8` image with the same height and width.
- The channel order of `lower` and `upper` **must match the image**. The image here is RGB, so the bounds are in R,G,B order. On a BGR image they'd have to be B,G,R.
- `np.array(..., dtype="uint8")` makes the bounds the same type as the image.
- `print(mask)` prints a large array of 0s and 255s.
- `cv2.bitwise_and(rgb, rgb, mask=mask)`:
    - ANDing an image with itself just returns the image,
    - but `mask=` limits the operation to pixels where the mask is non-zero, and everything else becomes **black**.
    - The result is the original colors, visible only where the color matched.
- The 3-panel plot shows the original, the mask, and the segmented image.

🎯 **For the project, do exactly this, but in HSV instead of RGB:**

- RGB thresholds break as soon as the lighting changes, because shadows change all three numbers.
- In HSV, the color lives in H (Session 2, part 3). So the pipeline is: convert to HSV, call `inRange` with an H range plus minimum S and V, and you have your flag mask.

Cells 7–8: histogram

```
gray = cv2.cvtColor(imgs, cv2.COLOR_BGR2GRAY)
hist = cv2.calcHist(gray, [0], None, [256], (0, 256))
plt.plot(hist)
```

- `cv2.calcHist(images, channels, mask, histSize, ranges)`:
    - `images` is a **list** of images.
    - `[0]` means "use channel 0".
    - `None` means "no mask, use the whole image".
    - `[256]` gives 256 bins, one per gray level.
    - `(0, 256)` is the value range, with the upper bound excluded.
- The x-axis is brightness (0–255) and the y-axis is how many pixels have it.
- 💡 Peaks show where the pixels cluster, for example a dark-background peak and a bright-coins peak. A good threshold sits in the **valley between them**.
- ⚠️ **Bug, which I confirmed by running it:** the code passes `gray` instead of `[gray]`. OpenCV then treats **each row as a separate image** and only counts the **first row**. Use `cv2.calcHist([gray], [0], None, [256], [0, 256])`. Cells 48–49 have the same bug.
- ⚠️ In your `ml_lab` environment (OpenCV 5), `calcHist` returns a **1-D** array, so `hist[:, 0]` in cells 48–49 raises `IndexError`. Use `hist.ravel()` instead.

Cells 9–12: binary thresholding

```
th,  thresh_img  = cv2.threshold(gray, 200, 255, cv2.THRESH_BINARY)
th2, thresh_img2 = cv2.threshold(gray, 200, 255, cv2.THRESH_BINARY_INV)
masked_img = cv2.bitwise_and(gray, gray, mask=thresh_img)
subtracted = cv2.subtract(gray, thresh_img)
```

- `cv2.threshold(src, thresh, maxval, type)` returns **two values**: the threshold actually used, and the output image.
- The input must be single-channel (gray).

Type	Pixel > thresh	Otherwise
`THRESH_BINARY`	`maxval` (255)	0
`THRESH_BINARY_INV`	0	`maxval`
`THRESH_TRUNC`	thresh	unchanged
`THRESH_TOZERO`	unchanged	0
`THRESH_TOZERO_INV`	0	unchanged

- The comparison is strictly greater (`>`), not `>=`.
- `masked_img` keeps the original gray values only where the pixel was brighter than 200.
- `cv2.subtract(gray, thresh_img)` is a **saturating** subtraction: where the mask is 255 the result is 0, so the bright objects are erased, and elsewhere the gray value is unchanged.
  - 💡 `cv2.subtract` clips at 0, while NumPy `a - b` on `uint8` **wraps around** (10 − 20 = 246). That's a big difference.
- The 2×3 plot shows the original, gray, the mask, the masked image, the subtracted image, and the inverted mask.

Cells 13–17: Otsu's automatic threshold

```
th, thresh_img_otsu = cv2.threshold(gray, 200, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
print(th)
```

- Adding `+ cv2.THRESH_OTSU` tells OpenCV to **choose the threshold itself**. The `200` is ignored, and `th` returns the value it chose.
- 💡 How it chooses: it tries every threshold and picks the one where the two groups (dark pixels and bright pixels) are each as **uniform as possible**. Formally, it minimizes the *within-class* variance, which is the same as maximizing the *between-class* variance.
  - ⚠️ The notebook says it makes "the variance between the background and foreground peaks minimal". That's backwards.
- It works well when the histogram has **two clear peaks** (bimodal), and badly when the lighting is uneven. Top-hat (below) fixes that case.

---

2. Morphological operations

💡 These operations work on the **shape** of white regions in a binary image (they also work on gray images). A small **structuring element** (kernel) slides over the image, like a stamp.

Cells 21–26: erosion

```
img = cv2.imread('Images/j.png', cv2.IMREAD_GRAYSCALE)
eroded = cv2.erode(img, None, iterations=1)
```

- **Rule:** a pixel stays white only if **every** pixel under the kernel is white. For gray images, the output is the **minimum** under the kernel.
- The effect: white shapes **shrink**, thin parts and small white specks **disappear**, and holes grow.
- `None` uses the default 3×3 square kernel.
- `iterations=n` repeats the operation n times, which is roughly the same as using a bigger kernel.

Cells 27–31: structuring elements

```
cv2.getStructuringElement(cv2.MORPH_RECT,    (5,5))   # full square of 1s
cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))   # disk-like
cv2.getStructuringElement(cv2.MORPH_CROSS,   (5,5))   # plus sign
kernel = np.ones((5, 5), np.uint8)                    # same as MORPH_RECT
```

- These cells print 5×5 arrays of 0 and 1 so you can **see** the shape.
- The size is given as `(width, height)`.

Cell 32: kernel shape matters

```
rect_kernel = cv2.getStructuringElement(cv2.MORPH_RECT,    [7, 7])
disk_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, [7, 7])
```

- When eroding circles, a square kernel makes them slightly **squarish**, while a disk kernel keeps them **round**.
- 💡 The kernel's shape gets "stamped" into the result. Use an ellipse for round, natural shapes.

Cells 33–36: dilation

```
dilated = cv2.dilate(img, None, iterations=1)
```

- **Rule:** a pixel becomes white if **any** pixel under the kernel is white (the **maximum**).
- The effect: white shapes **grow**, small black holes fill in, and nearby pieces **join**.
- This is the opposite of erosion. Cell 36 uses a bigger 11×11 kernel with rect vs disk, so the shape effect is easy to see.

Cells 37–40: opening = erode, then dilate

```
open_rect = cv2.morphologyEx(img, cv2.MORPH_OPEN, rect_kernel)
```

- `cv2.morphologyEx(src, op, kernel)` is the general function for compound operations.
- 💡 **Erosion** removes small white noise but also shrinks the real objects. **Dilation** then grows the survivors back to about their original size. The specks don't come back, because they no longer exist.
- Cell 39 (`j_noise`) shows the noise around the letter j disappearing.
- Cell 40 (`sticksandcircles` , 9×9 kernel) shows the **thin sticks removed and the circles kept**. Anything narrower than the kernel disappears, so opening can filter by **shape and size**.

Cells 41–43: closing = dilate, then erode

```
close_rect = cv2.morphologyEx(img, cv2.MORPH_CLOSE, rect_kernel)
```

- 💡 This is the mirror of opening. It **fills small black holes and gaps** inside white objects without making the objects bigger.
- Cell 43 uses a fingerprint with a `[4, 1]` kernel (**4 wide, 1 tall**), so it closes only **horizontal** gaps. Kernel shape can be directional.

Cells 44–45: close, then open

```
close_disk      = cv2.morphologyEx(img,        cv2.MORPH_CLOSE, disk_kernel)
open_close_disk = cv2.morphologyEx(close_disk, cv2.MORPH_OPEN,  disk_kernel)
```

- On the circuit image with an 11×11 disk: closing fills the gaps, then opening removes the thin traces, leaving only the big pads.
- Chaining operations is normal practice.

Cells 46–50: top-hat for uneven lighting

💡 **The problem:** in `rice.png`, the top is brighter than the bottom. A single global threshold such as 145 catches rice at the top but misses it at the bottom (cell 48).

```
rect_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, [50, 50])  # bigger than any rice grain
tophat_rect = cv2.morphologyEx(img, cv2.MORPH_TOPHAT, rect_kernel)
t, thresh_img = cv2.threshold(tophat_rect, 50, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

- Opening with a kernel **bigger than every grain** removes all the rice and leaves only the **background lighting**.
- **Top-hat = image − opening(image)**, which leaves the rice on a **flat** dark background. Otsu can then threshold it cleanly.
- ⚠️ The markdown says `Top-Hat = Image - Eroded_Image`. That's wrong: OpenCV's `MORPH_TOPHAT` is `image - opening`.
- **Cell 50 compares both:**

  ```
  eroded = cv2.morphologyEx(img, cv2.MORPH_ERODE, rect_kernel)
  tophat = img - eroded
  ```

  - `img - eroded` is safe in `uint8`, because erosion is a minimum and `eroded ≤ img` everywhere, so nothing wraps.
  - The eroded background is **darker** than the opened background (erosion ≤ opening), so `img - eroded` comes out brighter overall and isn't the true top-hat. Compare the two thresholded results in the plot.

🎯 Morphology in the project

- **Open** the flag mask to remove scattered pixels from the background that happen to match the color.
- **Close** it to fill holes in the flag caused by wrinkles, shadows or printing.

- Something like a 5×5 ellipse kernel for each is a good starting point.

---

3. Edge detection recap

Cells 53–56: load and blur

```
img = cv2.imread('./Images/input_image-1.jpg', 0)   # 0 = grayscale
img_blur = cv2.GaussianBlur(img, ksize=(3,3), sigmaX=1, sigmaY=1)
```

- Blurring first reduces noise before taking derivatives, which is Canny's stage 1 done by hand.

Cell 57: Sobel

```
sobelx  = cv2.Sobel(src=img_blur, ddepth=cv2.CV_8U, dx=1, dy=0, ksize=5)
sobely  = cv2.Sobel(src=img_blur, ddepth=cv2.CV_8U, dx=0, dy=1, ksize=5)
sobelxy = cv2.Sobel(src=img_blur, ddepth=cv2.CV_8U, dx=1, dy=1, ksize=5)
```

- ⚠️ `ddepth=cv2.CV_8U` clips negative gradients to 0, so **half the edges disappear** (the bright→dark ones). Use `cv2.CV_64F` and then `cv2.convertScaleAbs(...)`.
- ⚠️ `dx=1, dy=1` is **not** "combined X and Y". It's the mixed second derivative $\partial^2 I/\partial x \partial y$, which responds mostly to corners. The correct combined edge strength is the **magnitude** `√(gx² + gy²)` from Session 2, or approximately `cv2.addWeighted(abs_x, 0.5, abs_y, 0.5, 0)`.

Cells 58–61: Canny

```
edges = cv2.Canny(image=img_blur, threshold1=100, threshold2=200)
```

- See Session 2, part 5, for the 5 stages and the hysteresis thresholds.
- Cell 61 places Sobel and Canny side by side. Canny's edges are **thin, continuous and clean**.

---

4. Region-based segmentation: watershed

💡 Imagine the gray image as a **landscape**: bright areas are hills and dark areas are valleys. Put a water source (a **marker**) in each object and flood. Where the water from two markers meets, build a **dam**. The dams are the boundaries between objects. This separates **touching coins** that a plain threshold would merge into one blob.

Cell 63: threshold

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
ret, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
```

- Otsu picks the threshold, and `INV` makes the **coins white** and the background black.

Cell 64: finding sure regions

```
kernel  = np.ones((3,3), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2)  # remove noise
sure_bg = cv2.dilate(opening, kernel, iterations=3)                        # grow objects
dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
ret, sure_fg = cv2.threshold(dist_transform, 0.7*dist_transform.max(), 255, 0)
sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg)
```

- `opening` : cleans noise from the mask.
- `sure_bg` : after dilating, anything still **black** is *definitely background*.
- `distanceTransform(src, DIST_L2, 5)` : for each white pixel, the distance to the nearest black pixel. `DIST_L2` is ordinary straight-line distance, and `5` is the mask size. **Coin centers get the largest values.**
- Thresholding that at 70% of the max keeps only the coin **centers**. These are *definitely foreground*, and touching coins become **separate** blobs.
  - The final `0` is `cv2.THRESH_BINARY` written as a number.
  - The result is a float image, so it's converted to `uint8` .
- `unknown = sure_bg - sure_fg` is the ring around each coin center where the boundary must be. The watershed decides it.

Cell 65: labelling the markers

```
ret, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0
```

- `connectedComponents` gives each separate white blob its own number: 0 for the background, then 1, 2, 3 … N.
- `+1` shifts every label so the background becomes **1**, because watershed reserves **0** for "unknown, please decide".
- The unknown ring is then set to 0.

Cell 66: running the watershed

```
markers = cv2.watershed(img, markers)
img[markers == -1] = [255, 0, 0]
```

- `cv2.watershed(image, markers)` needs the **color (3-channel)** image and the int32 marker image. It returns the markers with the unknown pixels filled in and **boundaries marked as −1**.
- Boundary pixels are painted `[255,0,0]` . That's **blue** in BGR, but the plot uses matplotlib (RGB), so it appears **red**.
- ⚠️ `img` is modified **in place**, so re-run cell 63 before running this again.

💡 **For the project:** `cv2.connectedComponentsWithStats(mask)` is the upgraded version of `connectedComponents` . For **each blob** it gives the area, the bounding box and the **centroid**. That's exactly what's needed to find the biggest flag-colored blob, its center (for steering), and its area (for size).

Session 5 gotchas

1. `calcHist(gray, …)` → must be `calcHist([gray], …)`. Without the brackets, only the first row is counted.
2. OpenCV 5 `calcHist` returns 1-D, so `hist[:, 0]` fails; use `.ravel()`.
3. `inRange` bounds must use the same channel order as the image.
4. `cv2.subtract` saturates; NumPy `-` on `uint8` wraps.
5. Otsu **minimizes within-class** variance (the notebook's wording is backwards).
6. Top-hat is `image - opening`, not `image - eroded`.
7. `Sobel(..., CV_8U)` loses negative edges; `dx=1, dy=1` is not the combined edge.
8. `watershed` paints in BGR but the plot is RGB, and it modifies `img` in place.

🎯 The flag detector pipeline built from Sessions 2 and 5

1. Read a frame (Session 2, the camera loop).
2. `GaussianBlur` lightly (Session 2).
3. Convert BGR → HSV (Session 2).
4. Call `inRange` with the calibrated H/S/V range (Session 5, part 1). For red, OR two masks together.
5. **Open**, then **close** the mask (Session 5, part 2).
6. Call `connectedComponentsWithStats`, pick the **largest** blob, and ignore it if the area is too small (Session 5, part 4).
7. From its centroid and area, compute the offset from the image center and build the CSV line.